HeatWave Library Overview

000010343062

Document serial number: 000010343062

© 2016 Sony Interactive Entertainment Inc.
All Rights Reserved.
SIE Confidential

Table of Contents

1 Library Overview 3
Purpose and Characteristics 3
Features 3
Embedding into a Program 4
Samples 5
Reference Materials 5
2 Terminology 6
Listener 6
Sound Source 6
Sound 6
Sound Effect (Sfx) 7
Emitter Cone 7
Volume Roll-off 8
3 Using the Library 9
Initialization 9
Shutdown 9
Custom File and Memory Handlers 9
Coordinate System 9
Examples 9

000010343062

Document serial number: 000010343062

©SIE Inc

1 Library Overview

Purpose and Characteristics

HeatWave is a library to provide simple cross-platform 3D audio support. It is designed to provide a simple way to create sound effects for a game.

HeatWave supports the Win32, Win64, PlayStation®Vita, PlayStation®3 and PlayStation®4 platforms.

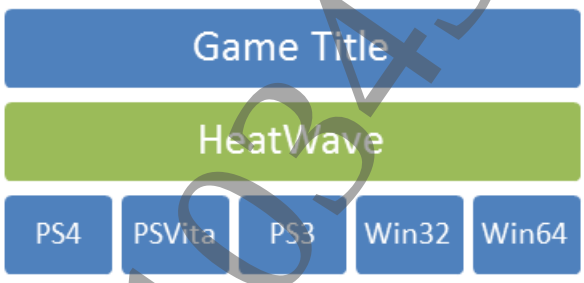
Features

The HeatWave library enables you to create 3D audio emitters and control a 3D audio listener.

The design of the API is intended to allow you to create and play sound effects in 3D with the minimum of programming and without the need to understand complex audio concepts. When you understand the basic concepts, playback of a HeatWave sound effect in your game is as easy as playing a simple audio file.

Figure 1 shows how the HeatWave library sits between platform-specific low-level audio libraries and your game, providing a simple, easy-to-use API layer without the complexity of some middleware solutions.

Figure 1 HeatWave Library Interaction



Supported Audio File Formats

HeatWave supports playback of the following audio formats:

- `.mp3` format audio files
- Uncompressed `.wav` format audio files with a bit depth of 16 bits

Audio data in unsupported formats will result in the `SCE_HEATWAVE_API_ERROR_UNSUPPORTED_FORMAT` error.

Streaming Audio

Streaming of large audio files is handled automatically by HeatWave. The library maintains an internal thread to maintain state and stream data as needed.

Thread Safety

HeatWave is multithread safe. It can be called from an interrupt handler.

Audio3d

HeatWave supports Audio3d for PlayStation®VR on the PlayStation®4 platform only. To enable Audio3d the library should be compiled with the `SCE_HEATWAVE_API_USE_AUDIO3D` define added to the pre-processor defines. HeatWave API functions will return

Document serial number: 000010343062

000010343062

SCE_HEATWAVE_API_ERROR_INVALID_OPERATION when Audio3d related functionality is used but support has not been enabled in the HeatWave library.

Default behavior is for audio to be played through the Audio3d bed. The HeatWave API also supports playback of flagged SceHwSfx as Audio3d objects. SceHwSfx flagged for playback as Audio3d objects must have a priority assigned. The priority is used internally when the number of requests exceed the number of available objects from the hardware. If an object is unable to be allocated the SceHwSfx will be rendered as part of the Audio3d bed.

Embedding into a Program

This section lists the files that are required in order to use HeatWave.

Table 1 Header Files Required for HeatWave

File Name	Description
include\heatwave_api.h	Main header file
include\heatwave_api_errors.h	Error return code header file
include\heatwave_api_types.h	API types header file

PlayStation®4

Table 2 Files Required for HeatWave on PlayStation®4

File Name	Description
lib\static\Orbis\libSceHeatWave.a	Static library file for PlayStation®4
libScePosix_stub_weak.a	System Posix library
libSceNgs2_stub_weak.a	NGS2 library file
libSceAudio3d_stub_weak.a	Audio3d library file
libSceSysmodule_stub_weak.a	System Module library file

PlayStation®Vita

Table 3 Files Required for HeatWave on PlayStation®Vita

File Name	Description
lib\static\PSVita\libSceHeatWave.a	Static library file for PlayStation®Vita

PlayStation®3

Table 4 Files Required for HeatWave on PlayStation®3

File Name	Description
lib\static\PS3\libSceHeatWave.a	Static library file for PlayStation®3
libpthread.a	System Posix library
libmstreamThreadMP3.a	MultiStream library file
libspurs_stub.a	SPURS library file
libaudio_stub.a	LibAudio library file
libsysutil_stub.a	SysUtil library file
libsysmodule_stub.a	SysModule library file

PlayStation®3 also requires the audio module to be loaded and initialized prior to calling sceHeatWaveInit(), as follows:

```
cellSysmoduleLoadModule(CELL_SYSMODULE_AUDIO);
cellAudioInit();
sceHeatWaveInit();
```

Document serial number: 000010343062

000010343062

Microsoft Windows

Table 5 Files Required for HeatWave on Microsoft Windows

File Name	Description
`lib\static\Win32\libSceHeatWave.lib`	Static library file for 32bit Windows
`lib\static\Win64\libSceHeatWave.lib`	Static library file for 64bit Windows
`mfuuid.lib`	Microsoft Media Foundation library
`mfplat.lib`	Microsoft Media Foundation library
`mfreadwrite.lib`	Microsoft Media Foundation library

Dynamic Libraries

The HeatWave package also includes shared versions of the library files to allow dynamic linkage. These versions of the library are available in `lib\shared`.

Samples

Sample programs using the HeatWave library can be found in `PhyreEngine\Samples\Audio`.

Reference Materials

- For more details about the HeatWave library, refer to the *HeatWave Library Reference*.
- For information about using audio with PhyreEngine™, refer to the *PhyreEngine™ Getting Started Guide*.
- For more details about Audio3d, refer to the *Audio3d Library Overview*.

000010343062

Document serial number: 000010343062

©SIE Inc

2 Terminology

This section defines some terms and concepts that you should understand before using the HeatWave library.

Listener

The listener object represents the player within the 3D audio scene. It has parameters for position, orientation and velocity. Typically the listener is set to the same position, velocity and rotation as the game camera. Velocity must not exceed the speed of sound, which is defined as 343.3 meters per second for the purposes of the HeatWave library. There is always one and only one listener.

The sounds that the (human) player hears depend on the relative positions, orientations and speeds of the listener and the various sound sources in the scene.

Sound Source

A sound source object generates audio data, for example by reading a `.wav` format audio file. Emitter cones and volume roll-off distances can be set per sound source.

Figure 2 A Source Supplies Audio Data

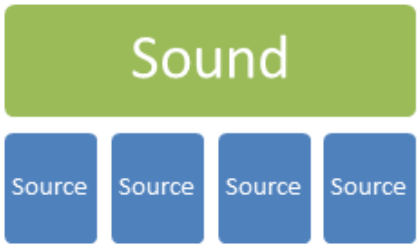


If 3D audio is not required, a sound source can be set up as 2D. For 2D sources, any 3D positional information is ignored, and you can control the stereo pan only. For example, you might define a 2D source for a background music soundtrack.

Sound

A sound object is a collection of sound sources that are triggered to play together with the same 3D positional information. For example, when a game character knocks on a door, you may want the player to hear a knock on wood and also the glass in the window rattling. Instead of creating a single `.wav` file of both noises happening at once, you can create a single sound in HeatWave that refers to two sound sources.

Figure 3 A Sound is a Collection of Sound Sources



0000103430 62

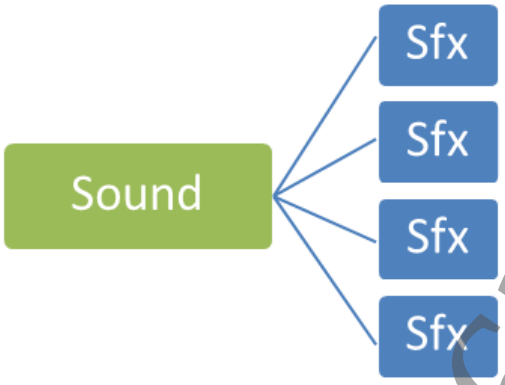
Document serial number: 000010343062

©SIE Inc

Sound Effect (Sfx)

A sound effect (Sfx) object is an instance of a sound used for playback. Multiple sound effects can be associated with a sound.

Figure 4 Multiple Sound Effects (Sfx) Can Point to a Single Sound



Each sound effect can be positioned in 3D space (location, orientation, and velocity). Sound effects can be set to play, pause, resume, and stop, and the volume and pitch of each effect can be modified independently.

In your game, you associate the sound effect with a trigger. For example, an enemy firing a gun should first set the sound effect location to the enemy's location, and then play the gunshot sound effect.

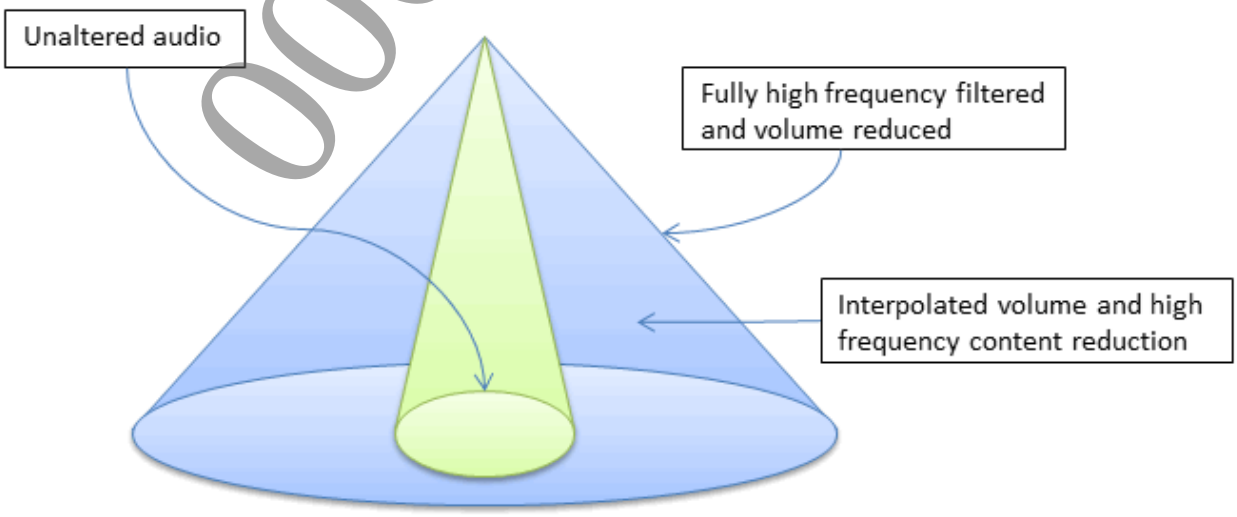
Note: The relative speed and distance of the sound effects and the listener in the scene are used to create the 3D sound and any Doppler shift.

Emitter Cone

An emitter cone specifies how the sound varies with direction; that is, whether the sound emanates in a tight cone from the emitter, or is more spread out.

An emitter cone varies the volume and high frequency content of the audio according to the angle of incidence between the source and the listener. Each source has parameters for the minimum angle (below which the audio is not altered) and the maximum angle (above which the audio is fully altered). There are also parameters to set the volume multiplier and the amount of high frequency filtering applied at the maximum angle. At angles between the minimum angle and the maximum angle, the volume and high frequency filter values are interpolated.

Figure 5 Emitter Cone Visualization



Document serial number: 000010343062

000010343062

©SIE Inc

The emitter cone can be altered for each source in the scene. This allows you to alter the behavior so that, for example, one sound source emits sound evenly all around, while another source is much more directional, and emits most of its sound in a tight cone.

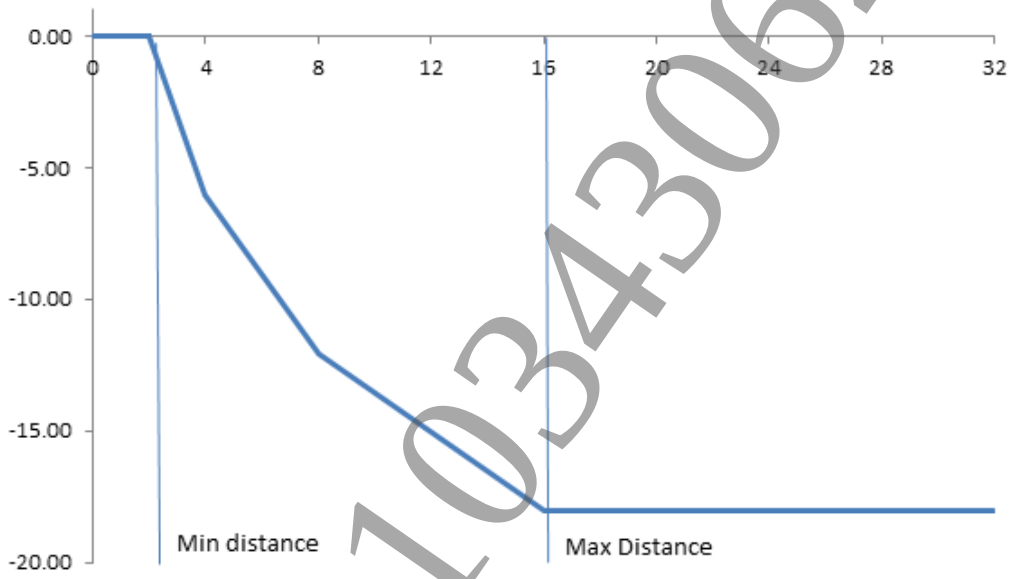
Volume Roll-off

Volume roll-off is the process of reducing the volume of a sound source in relation to its distance from the listener.

HeatWave uses an inverse square roll-off. This means that the volume halves with each doubling of the distance. The volume roll-off minimum and maximum distances can be set per source. The minimum distance is the distance at which volume attenuation begins to be applied. The maximum distance is the distance above which no further volume attenuation is applied.

The roll-off distance can be altered for each source in the scene.

Figure 6 Example of Volume (dB) Attenuation over Distance



000010343062

Document serial number: 000010343062

©SIE Inc

3 Using the Library

Initialization

To initialize the HeatWave library, call `sceHeatWaveInit()`. The function takes an argument for initialization parameters to be used.

Shutdown

To release all resources used by the HeatWave library and stop operation, call `sceHeatWaveRelease()`.

Custom File and Memory Handlers

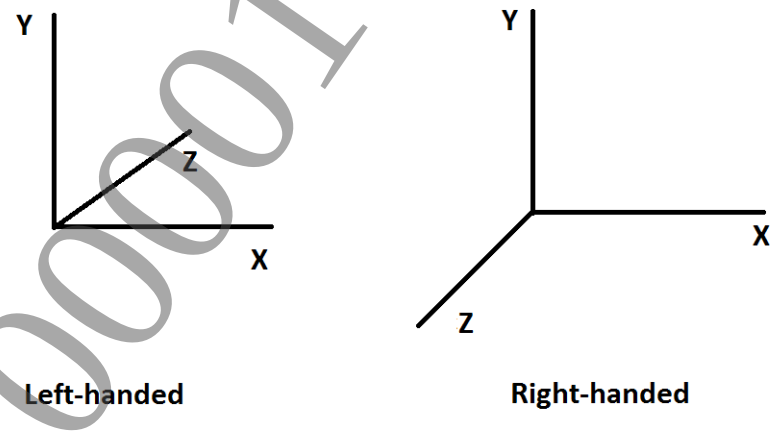
HeatWave uses default file-handling and memory allocation functions. However, it allows you to provide your own file- and memory-handling functions as callbacks.

- To override the default file handlers with user-specific versions, call `sceHeatWaveSetFileHandlers()`. The file `open`, `close`, `seek`, `tell` and `read` callbacks all need to be supplied when overriding the file handlers.
- To override the default memory allocation and freeing with user-specific versions, call `sceHeatWaveSetMemoryHandlers()`. Both memory allocation and freeing callbacks need to be supplied.

Coordinate System

All 3D coordinates in HeatWave are right-handed world space coordinates. If your game system uses the left-handed convention you need to convert coordinates and vectors by negating the Z-value before passing them to the HeatWave API. Figure 7 illustrates the two conventions.

Figure 7 Left- and Right-handed Coordinate Systems



Examples

Basic Playback

To playback a `.wav` format audio file:

(1) Initialize HeatWave by calling `sceHeatWaveInit()`.

(2) Create a `SceHwSound`.

(3) Create a `SceHwSource`.

000010343062

Document serial number: 000010343062

(4) Load an audio file into the `SceHwSource`.

(5) Get a `SceHwSfx` handle to the `SceHwSound`.

(6) Call play on the `SceHwSfx`.

The following code sample shows this process. The code comments refer to the steps listed above.

```
SceHwSound soundHandle;
SceHwSource sourceHandle;
SceHwSfx sfxHandle;
SceHwError res = SCE_HEATWAVE_API_OK;
const char *strFilePath = "PathToFile.wav";
SceHwInitParams initParams;
initParams.bLoopBack = false;

res = sceHeatWaveInit(&initParams); // Step 1
res = sceHeatWaveCreateSound(&soundHandle); // Step 2
res = sceHeatWaveSoundCreateSource(soundHandle, &sourceHandle); // Step 3
res = sceHeatWaveSourceLoadFile(sourceHandle, strFilePath); // Step 4
res = sceHeatWaveSoundCreateSfx(soundHandle, &sfxHandle); // Step 5
res = sceHeatWaveSfxPlay(sfxHandle); // Step 6
```

This code represents the simplest set of steps for creating and playing a sound in HeatWave. The following sections describe how to set locations for the listener and the sound effects, giving a more realistic game audio setup.

Playback with a Moving Listener

In the previous example, no information about the listener was set up, so HeatWave used a default listener. In this example, we create a listener and move it around:

```
SceHwSound soundHandle;
SceHwSource sourceHandle;
SceHwSfx sfxHandle;
float32_t sidestep = 0.10f;
SceHwVector position = { 0.0f, 0.0f,  1.0f };
SceHwVector velocity = { 0.0f, 0.0f,  0.0f };
SceHwVector forward  = { 0.0f, 0.0f, -1.0f };
SceHwVector up       = { 0.0f, 1.0f,  0.0f };
SceHwError res = SCE_HEATWAVE_API_OK;
const char *strPathToFile = "MyFile.wav";
SceHwSfxState state = kSfxState_Playing;

/* Create a sound */
res = sceHeatWaveCreateSound(&soundHandle);

/* Add a source to the sound */
res = sceHeatWaveSoundCreateSource(soundHandle, &sourceHandle);

/* Set the source to use the audio file */
res = sceHeatWaveSourceLoadFile(sourceHandle, strPathToFile);

/* Set the source to repeat */
res = sceHeatWaveSourceSetLoopCount(sourceHandle, knLoopInfinite);

/* Get a playback (sfx) handle for the sound */
res = sceHeatWaveSoundCreateSfx(soundHandle, &sfxHandle);

/* Play the Sfx */
res = sceHeatWaveSfxPlay(sfxHandle);

/* Move the listener back and forth horizontally */
while (state == kSfxState_Playing)
```

Document serial number: 000010343062

000010343062

```
{
    position.fX += sidestep;
    if ( (position.fX >= 10.0f) || (position.fX <= -10.0f) )
    {
        sidestep = -sidestep;
    }
    velocity.fX = sidestep * (1000.0f / 30.0f);

    /* Set listener data accordingly */
    res = sceHeatWaveSetListener(position, velocity, forward, up);

    /* Wait before next iteration... */

    res = sceHeatWaveSfxGetState(sfxHandle, &state);
}
```

As in the previous example, this code sample creates a sound, adds a source to the sound, and sets the source to use a given audio file. However, this time the source repeats. A Sfx/playback handle is created for the source, and then played. At this point, the sound is playing continually.

The sample then moves the listener around. The code to move the listener first checks whether the Sfx is playing. This will be true the first time through the loop; at the end of the loop the code checks the state by calling `sceHeatWaveSfxGetState()` for the `sfxHandle`.

The code then alters the position and velocity of the listener by calling `sceHeatWaveSetListener()`. This causes the listener to keep moving until the sound effect stops playing.

In this example, we specified a position for the listener and moved it around. However, we did not specify a position for the sound effect, so it is at the default location; that is, the origin. Therefore the listener was placed near the origin, pointing towards it, and then moved repeatedly to the left and right.

Moving Sound Effects in Space

The previous example showed how the listener can be moved. In a real game you might want to move sounds around too. The following code shows a simple example of this:

```
const int32_t kMaxSfx = 4;
SceHwSfx sfxHandles[kMaxSfx];
SceHwVector pos;
SceHwSound soundHandle;
SceHwSource sourceHandle;
SceHwError res = SCE_HEATWAVE_API_OK;
const char *strPathToFile = "MyFile.wav";

/* Create a sound */
res = sceHeatWaveCreateSound(&soundHandle);

/* Add a source to the sound */
res = sceHeatWaveSoundCreateSource(soundHandle, &sourceHandle);

/* Set the source to use the audio file */
res = sceHeatWaveSourceLoadFile(sourceHandle, strPathToFile);

/* Get playback (sfx) handles for the sound */
for (int32_t i = 0; i < kMaxSfx; i++)
{
    res = sceHeatWaveSoundCreateSfx(soundHandle, &sfxHandles[i]);
}

/* Place the sfx at various positions in the 3D world and play */
/* Positions are arbitrary */
pos.fX = -2.5f;
pos.fY = 1.0f;
```

Document serial number: 000010343062

```
pos.fZ = 1.0f;
res = sceHeatWaveSfxSetPosition(sfxHandles[0], pos);

pos.fX = 2.5f;
pos.fY = -2.0f;
pos.fZ = -1.0f;
res = sceHeatWaveSfxSetPosition(sfxHandles[1], pos);

pos.fX = 0.5f;
pos.fY = 5.0f;
pos.fZ = -4.0f;
res = sceHeatWaveSfxSetPosition(sfxHandles[2], pos);

pos.fX = 1.5f;
pos.fY = 1.0f;
pos.fZ = 5.0f;
res = sceHeatWaveSfxSetPosition(sfxHandles[3], pos);

for (int32_t i = 0; i < kMaxSfx; i++)
{
    res = sceHeatWaveSfxPlay(sfxHandles[i]);
}
```

As in the previous example, this code sample creates a single sound and sound source, and associates an audio file with the sound source. However, this time four sound effects are created for the sound, and each sound effect is set at a different position in space by calling `sceHeatWaveSfxSetPosition()` on each sound effect in turn.

The listener's position is not set in this sample, so it defaults to the origin.

000010343062

Document serial number: 000010343062

©SIE Inc